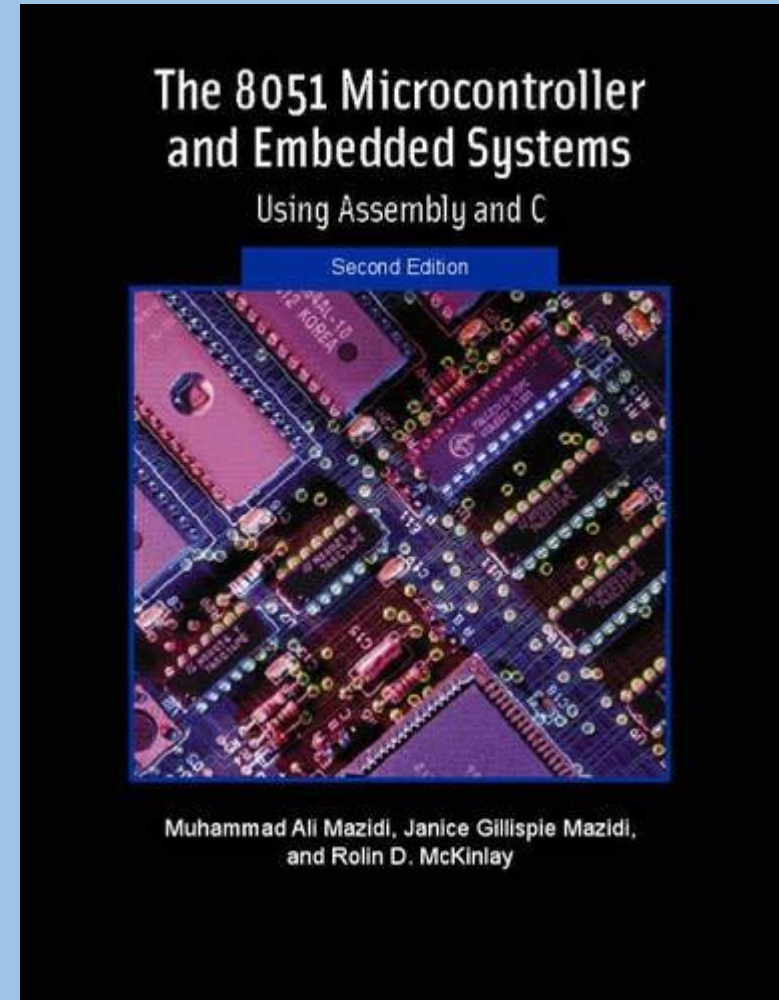


THE 8051 MICROCONTROLLER & Embedded Systems

Muhammad Ali Mazidi, Janice Mazidi
& Rolin McKinlay



JUMP, LOOP, AND CALL INSTRUCTIONS

Chapter 3

Objectives

Upon completion of this chapter, you will be able to:

- >> Code 8051 Assembly language instructions using loops
- >> Code 8051 Assembly language conditional jump instructions
- >> Explain conditions that determine each conditional jump instruction
- >> Code long jump instructions for unconditional jumps
- >> Code short jump instructions for unconditional short jumps
- >> Calculate target addresses for jump instructions
- >> Code 8051 subroutines
- >> Describe precautions in using the stack in subroutines
- >> Discuss crystal frequency versus machine cycle
- >> Code 8051 programs to generate a time delay

Objectives

- **Discussing the control transfer instructions available in 8051 Assembly language,**
- **Section 3.1, instructions used for looping, as well as instructions for conditional and unconditional jumps,**
- **Section 3.2, CALL instructions and their uses,**
- **Section 3.3, time delay subroutine.**

Example 3-1: Looping in the 8051

Write a program to

(a) clear ACC, then

(b) add 3 to the accumulator ten times.

Solution:

;This program adds value 3 to the ACC ten times

```
                MOV    A, #0          ;A=0, clear ACC
                MOV    R2, #10        ;load counter R2=10
AGAIN:          ADD    A, #03         ;add 03 to ACC
                DJNZ   R2, AGAIN      ;repeat until R2=0 (10 times)
                MOV    R5, A          ;save A in R5
```

Example 3-2

What is the maximum number of times that the loop in Example 3-1 can be repeated?

Solution:

Since R2 holds the count and R2 is an 8-bit register, it can hold a maximum of FFH (255 decimal); therefore, the loop can be repeated a maximum of 256 times.

Example 3-3: Loop Inside a Loop

Write a program to (a) load the accumulator with the value 55H, and (b) complement the ACC 700 times.

Solution:

Since 700 is larger than 255 (the maximum capacity of any register), we use two registers to hold the count. The following code shows how to use R2 and R3 for the count.

```
                MOV    A, #55H        ;A=55H
                MOV    R3, #10        ;R3=10, the outer loop count
NEXT:  MOV      R2, #70              ;R2=70, the inner loop count
AGAIN:  CPL     A                   ;complement A register
                DJNZ   R2, AGAIN      ;repeat it 70 times (inner loop)
                DJNZ   R3, NEXT
```

In this program, R2 is used to keep the inner loop count. In the instruction “DJNZ R2,AGAIN”, whenever R2 becomes 0 it falls through and “DJNZ R3,NEXT” is executed. This instruction forces the CPU to load R2 with the count 70 and the inner loop starts again. This process will continue until R3 becomes zero and the outer loop is finished.

Table 3-1: 8051 Conditional Jump Instructions

Instruction	Action
JZ	Jump if A = 0
JNZ	Jump if A $\neq$ 0
DJNZ	Decrement and jump if register $\neq$ 0
CJNE A, data	Jump if A $\neq$ data
CJNE reg, #data	Jump if byte $\neq$ #data
JC	Jump if CY = 1
JNC	Jump if CY = 0
JB	Jump if bit = 1
JNB	Jump if bit = 0
JBC	Jump if bit = 1 and clear bit

Example 3-4

Write a program to determine if R5 contains the value 0. If so, put 55H in it.

Solution:

```
MOV    A,R5        ;copy R5 to A
JNZ    NEXT        ;jump if A is not zero
MOV    R5,#55H
NEXT: . . .
```

Example 3-5

Find the sum of the values 79H, F5H, and E2H. Put the sum in registers R0 (low byte) and R5 (high byte).

Solution:

```

                                MOV    A, #0           ;clear A (A=0)
                                MOV    R5, A          ;clear R5
                                ADD     A, #79H        ;A=0+79H=79H
                                JNC     N_1            ;if no carry, add next number
                                INC     R5            ;if CY=1, increment R5
N_1:                            ADD     A, #0F5H       ;A=79+F5=6E and CY=1
                                JNC     N_2            ;jump if CY=0
                                INC     R5            ;If CY=1 then increment R5 (R5=1)
N_2:                            ADD     A, #0E2H       ;A=6E+E2=50 and CY=1
                                JNC     OVER           ;jump if CY=0
                                INC     R5            ;if CY=1, increment 5
OVER: MOV    R0, A          ;Now R0=50H, and R5=02
```

Example 3-6

Using the following list file, verify the jump forward address calculation.

Line	PC		Opcode	Mnemonic	Operand
01	0000			ORG	0000
02	0000	7800		MOV	R0 , #0
03	0002	7455		MOV	A , #55H
04	0004	6003		JZ	NEXT
05	0006	08		INC	R0
06	0007	04	AGAIN:	INC	A
07	0008	04		INC	A
08	0009	2477	NEXT:	ADD	A , #77h
09	000B	5005		JNC	OVER
10	000D	E4		CLR	A
11	000E	F8		MOV	R0 , A
12	000F	F9		MOV	R1 , A
13	0010	FA		MOV	R2 , A
14	0011	FB		MOV	R3 , A
15	0012	2B	OVER:	ADD	A , R3
16	0013	50F2		JNC	AGAIN
17	0015	80FE	HERE:	SJMP	HERE
18	0017				END

Example 3-6

Solution:

First notice that the JZ and JNC instructions both jump forward. The target address for a forward jump is calculated by adding the PC of the following instruction to the second byte of the short jump instruction, which is called the relative address. In line 4 the instruction “JZ NEXT” has opcode of 60 and operand of 03 at the addresses of 0004 and 0005. The 03 is the relative address, relative to the address of the next instruction INC R0, which is 0006. By adding 0006 to 3, the target address of the label NEXT, which is 0009, is generated. In the same way for line 9, the “JNC OVER” instruction has opcode and operand of 50 and 05 where 50 is the opcode and 05 the relative address. Therefore, 05 is added to 000D, the address of instruction “CLR A”, giving 12H, the address of label OVER.

Example 3-7

Verify the calculation of backward jumps in Example 3-6.

Solution:

In that program list, “JNC AGAIN” has opcode 50 and relative address F2H. When the relative address of F2H is added to 15H, the address of the instruction below the jump, we have $15H + F2H = 07$ (the carry is dropped). Notice that 07 is the address of label AGAIN. Look also at “SJMP HERE”, which has 80 and FE for the opcode and relative address, respectively. The PC of the following instruction, 0017H, is added to FEH, the relative address, to get 0015H, address of the HERE label ($17H + FEH = 15H$). Notice that FEH is -2 and $17H + (-2) = 15H$. For further discussion of the addition of negative numbers, see Chapter 6.

Example 3-8

Write a program to toggle all the bits of port 1 by sending to it the values 55H and AAH continuously. Put a time delay in between each issuing of data to port 1. This program will be used to test the ports of the 8051 in the next chapter.

Solution:

```

    ORG      0
BACK:  MOV    A,#55H      ;load A with 55H
      MOV    P1,A        ;send 55H to port 1
      LCALL  DELAY       ;time delay
      MOV    A,#0AAH     ;load A with AA (in hex)
      MOV    P1,A        ;send AAH to port 1
      LCALL  DELAY
      SJMP   BACK        ;keep doing this indefinitely
;----- this is the delay subroutine
      ORG    300H        ;put time delay at address 300H
DELAY: MOV    R5,#0FFH    ;R5 = 255 (FF in hex), the counter
AGAIN: DJNZ   R5,AGAIN    ;stay here until R5 becomes 0
      RET          ;return to caller (when R5 = 0)
      END           ;end of asm file
```

Example 3-9

Analyze the stack contents after the execution of the first LCALL in the following.

Solution:

```
001      0000          ORG      0
002      0000      7455  BACK:   MOV      A,#55H          ;load A with 55H
003      0002      F590          MOV      P1,A            ;send 55H to port 1
004      0004      120300        LCALL   DELAY           ;time delay
005      0007      74AA          MOV      A,#0AAH         ;load A with AAH
006      0009      F590          MOV      P1,A            ;send AAH to port 1
007      000B      120300        LCALL   DELAY
008      000E      80F0          SJMP    BACK             ;keep doing this
009      0010
010      0010      ;—————this is the delay subroutine
011      0300          ORG      300H
012      0300          DELAY:
013      0300      7DFF          MOV      R5,#0FFH         ;R5=255
014      0302      DDFE      AGAIN: DJNZ   R5,AGAIN        ;stay here
015      0304      22          RET                          ;return to caller
016      0305          END                                ;end of asm file
```

0A	
09	00
08	07
SP = 09	

Example 3-10

Analyze the stack for the first LCALL instruction in the following program.

[illegible]

Example 3-10

Solution:

First notice that for the PUSH and POP instructions we must specify the direct address of the register being pushed or popped. Here is the stack frame.

After the first LCALL			After PUSH 4			After PUSH 5		
0B			0B			0B	67	R5
0A			0A	99	R4	0A	99	R4
09	00	PCH	09	00	PCH	09	00	PCH
08	0B	PCL	08	0B	PCL	08	0B	PCL

**Figure 3-1. 8051
Assembly Main
Program That Calls
Subroutines**

```
;MAIN program calling subroutines
                                ORG      0
MAIN:    LCALL    SUBR_1
                                LCALL    SUBR_2
                                LCALL    SUBR_3

HERE:    SJMP     HERE
;-----end of MAIN
;
SUBR_1:   ....
          ....
          RET
;-----end of subroutine 1
;
SUBR_2:   ....
          ....
          RET
;-----end of subroutine 2

SUBR_3:   ....
          ....
          RET
;-----end of subroutine 3
          END                    ;end of the asm file
```

Example 3-11

A developer is using the Atmel AT89C1051 microcontroller chip for a product. This chip has only 1K byte of on-chip flash ROM. Which instruction, LCALL or ACALL, is more useful in programming this chip?

Solution:

The ACALL instruction is more useful since it is a 2-byte instruction. It saves one byte each time the call instruction is used.

Example 3-12

Rewrite Example 3-8 as efficiently as you can.

Solution:

```
                ORG      0
                MOV      A,#55H      ;load A with 55H
BACK:           MOV      P1,A        ;issue value in reg A to port 1
                ACALL    DELAY        ;time delay
                CPL      A            ;complement reg A
                SJMP     BACK         ;keep doing this indefinitely
```

;—————this is the delay subroutine

DELAY:

```
                MOV      R5,#0FFH    ;R5=255 (FF in hex), the counter
AGAIN:          DJNZ     R5,AGAIN     ;stay here until R5 becomes 0
                RET              ;return to caller
                END                ;end of asm file
```

Example 3-13

The following shows crystal frequency for three different 8051-based systems. Find the period of the machine cycle in each case.

- (a) 11.0592 MHz (b) 16 MHz (c) 20 MHz

Solution:

(a) $11.0592 \text{ MHz}/12 = 921.6 \text{ kHz}$; machine cycle is $1/921.6 \text{ kHz} = 1.085 \mu\text{s}$
(microsecond)

(b) $16 \text{ MHz}/12 = 1.333 \text{ MHz}$; machine cycle (MC) = $1/1.333 \text{ MHz} = 0.75 \mu\text{s}$

(c) $20 \text{ MHz}/12 = 1.66 \text{ MHz}$; MC = $1/1.66 \text{ MHz} = 0.60 \mu\text{s}$

Example 3-14

For an 8051 system of 11.0592 MHz, find how long it takes to execute each of the following instructions.

- (a) MOV R3,#55 (b) DEC R3 (c) DJNZ R2,target
(d) LJMP (e) SJMP (f) NOP (no operation) (g) MUL AB

Solution:

The machine cycle for a system of 11.0592 MHz is $1.085 \mu\text{s}$ as shown in Example 3-13. Table A-1 in Appendix A shows machine cycles for each of the above instructions. Therefore, we have:

Instruction	Machine cycles	Time to execute
(a) MOV R3,#55	1	$1 \times 1.085 \mu\text{s} = 1.085 \mu\text{s}$
(b) DEC R3	1	$1 \times 1.085 \mu\text{s} = 1.085 \mu\text{s}$
(c) DJNZ R2,target	2	$2 \times 1.085 \mu\text{s} = 2.17 \mu\text{s}$
(d) LJMP	2	$2 \times 1.085 \mu\text{s} = 2.17 \mu\text{s}$
(e) SJMP	2	$2 \times 1.085 \mu\text{s} = 2.17 \mu\text{s}$
(f) NOP	1	$1 \times 1.085 \mu\text{s} = 1.085 \mu\text{s}$
(g) MUL AB	4	$4 \times 1.085 \mu\text{s} = 4.34 \mu\text{s}$

Example 3-15

Find the size of the delay in the following program, if the crystal frequency is 11.0592 MHz.

```
                MOV      A,#55H  ;load A with 55H
AGAIN:  MOV      P1,A           ;issue value in reg A to port 1
                ACALL  DELAY     ;time delay
                CPL      A       ;complement reg A
                SJMP    AGAIN    ;keep doing this indefinitely
;----Time delay
DELAY:  MOV      R3,#200        ;load R3 with 200
HERE:   DJNZ     R3,HERE        ;stay here until R3 become 0
                RET             ;return to caller
```

Solution:

From Table A-1 in Appendix A, we have the following machine cycles for each instruction of the DELAY subroutine.

		Machine Cycle
DELAY:	MOV R3,#200	1
HERE:	DJNZ R3,HERE	2
	RET	2

Therefore, we have a time delay of $[(200 \times 2) + 1 + 2] \times 1.085 \mu\text{s} = 436.255 \mu\text{s}$.

Example 3-16

For an 8051 system of 11.0592 MHz, find the time delay for the following subroutine:

	Machine Cycle
DELAY: MOV R3,#250	1
HERE: NOP	1
NOP	1
NOP	1
NOP	1
DJNZ R3,HERE	2
RET	2

Example 3-16

Solution:

The time delay inside the HERE loop is $[250(1 + 1 + 1 + 1 + 2)] \times 1.085 \mu\text{s} = 1500 \times 1.085 \mu\text{s} = 1627.5 \mu\text{s}$. Adding the two instructions outside the loop we have $1627.5 \mu\text{s} + 3 \times 1.085 \mu\text{s} = 1630.755 \mu\text{s}$.

If machine cycle timing is critical to your system design, make sure that you check the manufacture's data sheets for the device specification. For example, the DS89C430 has 3 machine cycles instead of 2 machine cycles for the RET instruction.

Example 3-17

For a machine cycle of 1.085 ms, find the time delay in the following subroutine.

DELAY:		Machine Cycle
	MOV R2,#200	1
AGAIN:	MOV R3,#250	1
HERE: NOP		1
	NOP	1
	DJNZ R3,HERE	2
	DJNZ R2,AGAIN	2
	RET	2

Example 3-17

Solution:

For the HERE loop, we have $(4 \times 250) \times 1.085 \mu\text{s} = 1085 \mu\text{s}$. The AGAIN loop repeats the HERE loop 200 times; therefore, we have $200 \times 1085 \mu\text{s} = 217000$, if we do not include the overhead. However, the instructions “MOV R3,#250” and “DJNZ R2,AGAIN” at the beginning and end of the AGAIN loop add $(3 \times 200 \times 1.085 \mu\text{s}) = 651 \mu\text{s}$ to the time delay. As a result we have $217000 + 651 = 217651 \mu\text{s} = 217.651$ milliseconds for total time delay associated with the above DELAY subroutine. Notice that in the case of a nested loop, as in all other time delay loops, the time is approximate since we have ignored the first and last instructions in the subroutine.

Table 3-2: Clocks per Machine Cycle (MC) for Various 8051 Versions

Chip/Maker	Clocks per Machine Cycle
AT89C51 Atmel	12
P89C54X2 Philips	6
DS5000 Dallas Semi	4
DS89C430/40/50 Dallas Semi	1

Example 3-18

From Table 3-2, find the period of the machine cycle (MC) in each case if XTAL = 11.0592 MHz, and discuss the impact on performance.

(a) AT89C51 (b) P89C54X2 (c) DS5000 (d) DS89C4x0

Solution:

(a) $11.0592 \text{ MHz}/12 = 921.6 \text{ kHz}$; MC is $1/921.6 \text{ kHz} = 1.085 \mu\text{s}$ (microsecond) = 1085 ns

(b) $11.0592 \text{ MHz}/6 = 1.8432 \text{ MHz}$; MC is $1/1.8432 \text{ MHz} = 0.5425 \mu\text{s} = 542 \text{ ns}$

(c) $11.0592 \text{ MHz}/4 = 2.7648 \text{ MHz}$; MC is $1/2.7648 \text{ MHz} = 0.36 \mu\text{s} = 360 \text{ ns}$

(d) $11.0592 \text{ MHz}/1 = 11.0592 \text{ MHz}$; MC is $1/11.0592 \text{ MHz} = 0.0904 \mu\text{s} = 90 \text{ ns}$

Table 3-3: Comparison of 8051 and DS89C4x0 Machine Cycles

Instruction	8051	DS89C4x0
MOV R3,#value	1	2
DEC Rx	1	1
DJNZ	2	4
LJMP	2	3
SJMP	2	3
NOP	1	1
MUL AB	4	9

Example 3-19

For an AT8051 and DS89C430/40/50 system of 11.0592 MHz, find how long it takes to execute each of the following instructions.

- (a) MOV R3,#55 (b) DEC R3 (c) DJNZ R2,target (d) LJMP (e) SJMP
(f) NOP (no operation) (g) MUL AB

Solution:

Refer to example 3-18 and table 3-3 .

Instruction	AT8051	DS89C430/40/50
(a) MOV R3,#55	1x1085 ns = 1085 ns	2x90 ns = 180 ns
(b) DEC R3	1x1085 ns = 1085 ns	1x90 ns = 90 ns
(c) DJNZ R2,..	2x1085 ns = 2170 ns	4x90 ns = 360 ns
(d) LJMP	2x1085 ns = 2170 ns	3x90 ns = 270 ns
(e) SJMP	2x1085 ns = 2170 ns	3x90 ns = 270 ns
(f) NOP	1x1085 ns = 1085 ns	1x90 ns = 90 ns
(g) MUL AB	4x1085 ns = 4340 ns	9x90 ns = 810 ns

Example 3-20

Find the time delay for the loop section of the following subroutine if it is run on a DS89C430 chip, assuming a crystal frequency of 11.0592 MHz.

DS89C430 Machine Cycle

DELAY: MOV R3,#250

HERE: NOP	1
NOP	1
NOP	1
NOP	1
DJNZ R3,HERE	4
RET	

Solution:

The time delay inside the HERE loop is $[250(1 + 1 + 1 + 1 + 4)] \times 90 \text{ ns} = 2000 \times 90 \text{ ns} = 180 \text{ } \mu\text{s}$. Comparing this with Example 3-16, we see DS89C4x0 is about 9 times faster. $(1627 \text{ } \mu\text{s} / 180 \text{ } \mu\text{s} = 9)$

Example 3-21

Write a program to toggle all the bits of P1 every 200 ms. Assume that the crystal frequency is 11.0592 MHz and that the system is using the AT89C51.

Solution:

;Tested for AT89C51 of 11.0592 MHz.

```
                MOV     A,#55H
AGAIN:          MOV     P1,A
                ACALL   DELAY
                CPL      A
                SJMP     AGAIN

;-----Time delay
DELAY:          MOV     R5,#2
HERE1:          MOV     R4,#180
HERE2:          MOV     R3,#255
HERE3:          DJNZ    R3,HERE3
                DJNZ    R4,HERE2
                DJNZ    R5,HERE1
                RET
```

$$2 \times 180 \times 255 \times 2 \text{ MC} \times 1.085 \mu\text{s} = 199,206 \mu\text{s}$$

Example 3-22

Write a program to toggle all the bits of P1 every 200 ms. Assume crystal frequency is 11.0592 MHz and the system is using DS89C430/40/50.

Solution:

```
;Tested for DS89C430 of 11.0592 MHz.
```

```
                MOV     A,#55H
AGAIN:  MOV     P1,A
                ACALL  DELAY_200m
                CPL     A
                SJMP   AGAIN
```

```
;----Time delay
```

```
DELAY_200m:
```

```
                MOV     R5,#9
HERE1:  MOV     R4,#242
HERE2:  MOV     R3,#255
HERE3:  DJNZ    R3,HERE3
                DJNZ    R4,HERE2
                DJNZ    R5,HERE1
                RET
```

Delay $9 \times 242 \times 255 \times 4 \text{ MC} \times 90 \text{ ns} = 199,940 \mu\text{s}$